

Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

КУРСОВА РОБОТА
з дисципліни ”Об’єктно-орієнтоване програмування”
на тему: «РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ ДЛЯ МОДЕЛЮВАННЯ
ОБ’ЄКТІВ ”Roman Conquest. Макрооб’єкти: Джерело руди, База, Башта. Мікрооб’єкти: Воїн,
Центуріон, Преторіанець”
З ВИКОРИСТАННЯМ UML ТА МОВИ ПРОГРАМУВАННЯ JAVA»

Виконав: студент 1-го курсу групи 2ПІ-256
спеціальності F2 Інженерія програмного
забезпечення
Олександр КОВАЛЬ

СЛ
Signature

Керівник доцент кафедри ПЗ
к.т.н., доц. Олександр РЕШЕТНИК

Кількість балів: ##
Оцінка: ЄКТС #

Члени комісії:

(Подпись)

Решетнік О.О.

(подпис)

Катєльніков Д.І.

(підпис)

(прізвище та ініціали)

м. Вінниця - 2026 рік

Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення
Рівень вищої освіти перший (бакалаврський)
Спеціальність F2 Інженерія програмного забезпечення
Освітньо-професійна програма «Інженерія програмного забезпечення»



ЗАТВЕРДЖУЮ

Завідувач кафедри ПЗ

д.т.н., проф., О.Н. Романюк

«10» лютого 2026 року

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ

на курсову роботу з дисципліни "Об'єктно-орієнтоване програмування"

Ковалю Олександрю Дмитровичу, студенту гр. 2ПІ-256

Тема роботи: РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ ДЛЯ МОДЕЛЮВАННЯ ОБ'ЄКТІВ "Roman Conquest.

Макрооб'єкти: Джерело руди, База, Башта. Мікрооб'єкти: Воїн, Центуріон, Преторіанець" З ВИКОРИСТАННЯМ UML ТА МОВИ ПРОГРАМУВАННЯ JAVA.

Завдання: розробити програмну систему і пояснювальну записку до процесу розробки.

Вхідні дані до виконання проєкту:

Мова програмування: Java.

Середовище розробки: NetBeans IDE, IntelliJ IDEA, Eclipse або Visual Studio Code.

Графічний інтерфейс: платформа JavaFX, Swing або AWT.

Зображення мікрооб'єкта: мінімум три графічних примітива (один з яких – текст). Зображення кожного макрооб'єкту повинно складатись не менше чим з трьох графічних примітивів (лінія, прямокутник, коло, арка, полігон, текст, графічний образ та інших), при цьому обов'язково один з примітивів повинен бути текст. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання. Для класу

мікрооб'єкта слід реалізувати глибинне копіювання. Макроб'єкти повинні мати можливість містити кількість мікрооб'єктів більше двох. Повина бути реалізована ієрархія класів мікрооб'єктів, яка містить як мінімум три рівня наслідування, які можуть інстанціюватись і відображатись на екрані. Повинен бути реалізований наступний запит: знайти мікрооб'єкт з вказаними параметрами. Програма шукає об'єкт з введеними параметрами і видає на екран: його місце розташування, приналежність до макроб'єктів (чи зазначає, що мікрооб'єкт не належить жодному макроб'єкту). Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які належать вказаному макроб'єкту. Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які не належать жодному макроб'єкту. Повинен бути реалізований наступний запит: встановити критерій сортування мікрооб'єктів у запитах, які виводять переліки. Критеріїв сортування повинно бути не менше трьох, наприклад: Ім'я_студента, Середній_бал, Кількість_Оцінок_4_та_вище. Повинен бути реалізований автоматичний рух деяких мікрооб'єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою; 2) деякі мікрооб'єкти заходять в макроб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Необхідно реалізувати щонайменше три запити на підрахунок мікрооб'єктів. Наприклад: а) кількість активних мікрооб'єктів; б) кількість об'єктів із рівнем життя понад 50%; в) кількість об'єктів, що розташовані у правій частині екрана тощо. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач. Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) на мінікарті повинна бути позначена видима область універсального об'єкта; 2) рух видимої області універсального об'єкта відповідно змінює мінікарту; 3) макроб'єкти та мікрооб'єкти позначені на мінікарті; 4) рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті; 5) натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на екрані.

Зміст пояснювальної записки:

АНОТАЦІЯ

ЗМІСТ

ВСТУП

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ
ЗАВДАННЯ НА РОБОТУ

2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

4 КЕРІВНИЦТВО КОРИСТУВАЧА

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

ДОДАТОК А (обов'язковий) Лістинг програми

Дата видачі завдання: "11" лютого 2026 р.

Строк подання завершеної курсової роботи: 18 червня 2026 р.

Керівник _____

Signature
(підпис)

Олександр РЕШЕТНІК

Завдання отримав _____

Signature
(підпис)

Олександр КОВАЛЬ

АНОТАЦІЯ

Коваль О.Д. Розробка програмної системи для моделювання об'єктів "Roman Conquest. Макрооб'єкти Джерело руди, База, Башта. Мікрооб'єкти: Воїн, Центуріон, Преторіанець." з використанням UML та мови програмування Java. : курсова робота з дисципліни «Об'єктно-орієнтоване програмування»

У курсовій роботі здійснена розробка програмного застосунку для ОС Windows/Linux/Mac OS, виконаний за допомогою JavaFX. Програмний застосунок моделює функціонування об'єктів "воїни, будівлі" з використанням UML та мови програмування Java. Використано усі основні принципи об'єктно-орієнтованого програмування, а саме: абстракція, інкапсуляція, успадкування та поліморфізм. Проаналізовано проблему зменшення зацікавленості в історії стародавнього Риму та інфраструктуру його армії. Створено готовий програмний продукт.

Ключові слова: мікрооб'єкти, успадкування, макрооб'єкти, агрегація, універсальний об'єкт, композиція.

ЗМІСТ

ВСТУП	4
1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ	7
1.1 Предметна область	7
1.2 Існуючі реалізації	10
1.3 Розробка технічного завдання на роботу	12
1.4 Обґрунтування вибору мови програмування	13
1.5 Висновки	17
2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ	18
2.1 Модель графічного відображення	20
2.2 Графічні процедури підсистеми графічного відображення	22
3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ	24
3.1 Діаграма наслідування	24
3.2 Діаграма агрегації	26
3.3 Діаграма композиції	26
3.4 Діаграма асоціації	26
3.5 Діаграма кооперації	30
3.6 Діаграма послідовності	30
3.7 Діаграма стану	32
4 КЕРІВНИЦТВО КОРИСТУВАЧА	34
ВИСНОВКИ	37
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	38
ДОДАТОК А (обов'язковий) Лістинг програми	39

ВСТУП

Сучасний етап розвитку людської цивілізації нерозривно пов'язаний із глобальною цифровізацією та стрімким впровадженням інноваційних технологій у всі сфери нашої життєдіяльності. Досягнення сучасних інформаційних технологій (ІТ) здійснили справжню революцію з точки зору збереження інформації, її передачі на великі відстані, математичного та логічного обрахунку у реальному часі, а також високоякісної візуалізації, в тому числі тривимірної. Людство здійснило колосальний стрибок від розрізнених паперових архівів до багаторівневих реляційних та об'єктно-орієнтованих баз даних, хмарних сховищ, які здатні надійно акумулювати терабайти історичних, статистичних та структурних відомостей, гарантуючи їхню стійкість до втрат. Розвиток глобальних оптоволоконних та бездротових мереж дозволяє обмінюватися цими масивами даних практично миттєво, стираючи будь-які географічні межі. Проте одним із найважливіших досягнень є здатність сучасних обчислювальних систем та парадигм програмування (зокрема об'єктно-орієнтованого підходу) обробляти складні алгоритми у реальному часі. Використання багатопотоковості дозволяє системі одночасно керувати станами тисяч незалежних програмних сутностей. У поєднанні з передовими графічними рушіями, це дає змогу відтворювати гіперреалістичні симуляційні світи з деталізованою фізикою, де кожен елемент функціонує за наперед заданими правилами.

Незважаючи на цей безпрецедентний технологічний прогрес, сучасне суспільство стикається з глибокою соціокультурною та освітньою кризою, що набуває ознак глобальної проблеми. Складається враження, що люди починають все менше звертати увагу на історію Давнього Риму, його фундаментальну культуру та неоціненний внесок у формування сучасної європейської та світової цивілізації. Світ охоплюють своєрідні інформаційні жахи сьогодення: молоде покоління стрімко втрачає історичну пам'ять, страждає від так званого «кліпового мислення» та повністю розсіює власну увагу в нескінченних потоках низькоякісного, швидкого розважального контенту. Суспільство поступово

забуває, що Давній Рим славився не тільки видатними філософами, талановитими архітекторами, правознавцями і науковцями, а й, на свій час, найбільш розвинутою та ефективною мілітаризованою сферою. Це була система з надзвичайно чіткою, бездоганно налагодженою військовою та суспільною ієрархією, залізною дисципліною та передовим стратегічним плануванням, яке дозволило побудувати одну з найвеличніших імперій в історії. Державні інституції та традиційні освітні заклади продовжують нести величезні витрати коштів на друк консервативних підручників та підтримку застарілих методик викладання. Проте ці колосальні фінансові вливання не дають бажаного результату, викликаючи у сучасній аудиторії лише нудьгу та відторгнення. Це реальна проблема, яку неможливо подолати класичними інструментами, але яку можна успішно вирішити шляхом інтерактивного комп'ютерного моделювання і прогнозування, адаптувавши подачу інформації до вимог цифрової епохи.

Можливість застосування сучасних ІТ у проблемі, що розглядається, відкриває абсолютно нові, безмежні горизонти для інноваційних освітніх підходів. Замість пасивного споживання сухого текстового матеріалу, ми маємо унікальну можливість використати всю потужність об'єктно-орієнтованого програмування для створення інтерактивної форми ознайомлення. Розробка динамічної стратегічної гри або навчального симулятора дозволяє перевести складний історичний контекст у зрозумілу та захопливу площину. Програмне середовище надає інструментарій для детального моделювання архітектури, економіки та життєдіяльності римського військового табору. З інфраструктурної точки зору, така програмна система органічно спирається на глобальні макрооб'єкти: «Джерело руди» забезпечуватиме економічне підґрунтя та критично важливу ресурсну базу для розвитку, центральна «База» відповідатиме за логістику, управління та генерацію нових одиниць, а «Башта» формуватиме неприступний оборонний периметр та надаватиме тактичну перевагу. З іншого боку, деталізація ігрових процесів реалізовуватиметься через мікрооб'єкти, які ідеально репрезентують римську військову ієрархію: від базового «Воїна» (легіонера), через тактично підготовленого «Центуріона», і аж до елітного «Преторіанця». Користувач отримує

можливість безпосередньо взаємодіяти з цією багаторівневою системою, керувати розподілом ресурсів, розробляти власні стратегії та аналізувати наслідки прийнятих рішень. Через такий захопливий ігровий процес молодь підсвідомо засвоює складні принципи римської організації, що робить вивчення історії максимально ефективним та живим.

З огляду на нагальну необхідність подолання глибокої кризи історичної необізнаності, а також враховуючи колосальний потенціал сучасних інструментів об'єктно-орієнтованого програмування для створення інтерактивних навчально-ігрових симуляцій, саме тому в якості теми курсової роботи було обрано розробку програмної системи моделювання об'єктів гри Roman Conquest.

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ

1.1 Предметна область

Давній Рим залишається однією з найвпливовіших цивілізацій в історії людства, чий досягнення у військовій справі, державному управлінні та культурі заклали фундамент сучасної європейської цивілізації. Римська армія вважається одним із найдосконаліших військових механізмів античності, що забезпечило створення і підтримку однієї з найбільших імперій у світовій історії. Проте сучасне покоління все менше цікавиться історією, особливо її військовими аспектами, віддаючи перевагу швидкому розважальному контенту замість глибокого вивчення історичних подій. Сучасні технології відкривають нові можливості для вивчення та популяризації історії через інтерактивні графічні програми, які дозволяють у захоплюючій формі демонструвати взаємодію макро- та мікрооб'єктів один з одним.

Військова організація Давнього Риму базувалася на чіткій ієрархічній структурі, де кожен воїн мав своє місце та функції. Кожна війна у античному світі не відбувалась без **воїнів (Warrior)**, які складали основу легіонів. Згідно з відомим висловом Отто фон Бісмарка, війни виграють не лише генерали, а й звичайні люди, які беруть у них участь. У римській армії воїни часто приходили на військову службу не з власного бажання, а через необхідність захищати інтереси Римської імперії на численних фронтах. Ці люди були змушені піти на війну і битися за Римську Імперію, формуючи хребет римської військової машини. Воїни брали участь у кампаніях на Заході та Сході, воювали з галами в глибинах Іспанії, переправлялися через Середземне море для боротьби з Карфагеном. Саме вони виконували накази командирів та брали участь у найважчих битвах.

Після багатьох років служби та участі в численних кампаніях, коли воїн провів достатньо військових операцій і здобув необхідний досвід та гроші, найздібніші з них могли отримати підвищення у ієрархії римського легіону і стати

центуріонами (Centurio). Це звання походить від латинського слова, що означає десяток. Ця людина отримувала у свою владу вже десяток людей і повагу, краще спорядження та титул. Центуріони відмітилися в історії Риму як вправні воїни та мудрі командувачі, вони є надважливою ланкою у ході бою, на плечах яких лежав величезний обсяг завдань. Центуріони відігравали ключову роль у римській армії, адже на них лежала відповідальність за підготовку солдатів, підтримання дисципліни та прийняття тактичних рішень безпосередньо на полі бою. Вони отримували більшу повагу з боку товаришів та вищу оплату. Їхня майстерність володіння мечем та пілумом, стратегічне мислення та лідерські якості робили їх незамінними у військовій ієрархії.

Як тільки центуріон відзначився у бою, зробив героїчний вчинок, став найкращим майстром меча та пілума у своєму легіоні або врятував своїх людей від безвихідного оточення, йому пропонували стати **преторіанцем (Praetorian)**. Найвищою ланкою в ієрархії римських воїнів були преторіанці. Цих елітних воїнів відбирали серед найкращих солдатів з усієї імперії, і їхня задача була найскладнішою – захищати консулів та імператора. Преторіанці – це воїни при імператорі, це його особиста армія, розташована безпосередньо в Римі. Бути преторіанцем означало мати значний титул і перебувати у відносній безпеці навіть протягом довгих воєн, тому в програмі на тему Roman Conquest вони є найвищою ланкою ієрархії. Преторіанці отримували значно вищу платню порівняно зі звичайними легіонерами та мали особливі привілеї. Також, преторіанці були особливими воїнами, захисниками імператора, останнім етапом опору, котрий відділяв і обороняв сенаторів від населення.

Проте успіх будь-якої імперії залежить не лише від сили її армії, а й від економічної основи. Кожна імперія була побудована насиллям, проте для існування імперії, утримання регулярної армії та побудови нових колоній необхідні гроші. Римська імперія була побудована на системі ресурсного забезпечення, де ключову роль відігравали рудники. **Джерело руди (Source)** є спорудою та макрооб'єктом, що приносить руду до команди гравця. Воно може бути захоплене будь-якою командою, захопити його може будь-яка ланка із ієрархії мікрооб'єктів, але в

залежності від навичок конкретного воїна час захоплення може змінюватися. Контроль над джерелами руди означав контроль над виробництвом зброї, інструментів та монет, необхідних для функціонування держави. Ці стратегічні об'єкти часто ставали предметом конфліктів між різними фракціями.

На шляху до бази імперії зустрічаються перешкоди. Тому в якості інтерпретації цього в програмі було обрано **башту (Tower)** як макрооб'єкт, що являє собою перешкоду для ворожої армії, яку необхідно зруйнувати щоб отримати шлях до бази супротивника. Оборонні башти виконували кілька важливих функцій. По-перше, вони забезпечували захищену позицію для оборонців. По-друге, башти слугували пунктами спостереження, звідки можна було виявити наближення ворога. По-третє, вони використовувалися як бази для постачання. Башти – це споруди, де мікрооб'єкти-союзники можуть отримати допомогу у вигляді поповнення очок здоров'я, також вони атакуватимуть найближчого ворожого мікрооб'єкта в певній області. Водночас ворожі башти становили загрозу, оскільки їхні захисники могли атакувати римських солдатів. Важливою особливістю башт є можливість полагодити їх, але зробити це може тільки центуріон завдяки набутому досвіду в ході кампаній та застосуванню його в реаліях битви.

Центральним елементом військової організації була **база (Base)** або головний табір. База є головною спорудою гравця, її треба захищати будь-якою ціною, оскільки база є початковою точкою всіх мікрооб'єктів будь-якої команди. Римські військові табори були справжніми шедеврами інженерної думки, де кожен елемент мав своє призначення. База служила відправною точкою для всіх військових операцій, місцем підготовки новобранців та центром командування. Втрата головної бази означала катастрофу для римського легіону, тому її захист був пріоритетним завданням. У той же час знищення ворожої бази було головною стратегічною метою в будь-якій військовій кампанії. Знищити базу – головна мета ворожої команди в програмі на тему Roman Conquest.

Ефективність римської армії значною мірою базувалася на взаємодії між різними рангами військовослужбовців. Як і в реальному житті, на війні панує співпраця між воїнами. Звичайні воїни не мають жодних особливих здібностей,

проте вони, перебуваючи поруч із досвідченими преторіанцями, дивлячись на них, можуть отримати натхнення до битви, бути мотивованими, отримуючи підвищення своїх характеристик поки перебувають у певній близькості з преторіанцем. Такий психологічний фактор відігравав важливу роль у підтриманні бойового духу. Центуріони, як зазначалось вище, можуть лагодити башти, щоб ті ще трохи допомогли у ході конфлікту. Ця система наставництва працювала природним чином та давала відчутні результати у бою. Сучасні технології дають змогу моделювати ці взаємодії через об'єктно-орієнтоване програмування, поєднуючи вивчення історії з демонстрацією принципів ООП.

1.2 Існуючі реалізації

На сьогодні існує чимало програмних продуктів, що присвячені тематиці Давнього Риму та його військовій історії. На цю тему є багато програмних реалізацій, проте кожна з них має свої особливості, переваги та недоліки. Кожна гра відображає різні аспекти стародавньої держави, проте всі ігри мають перелік проблем, що не дозволяють вважати їх вдалим вибором для популяризації ідеї вивчення історію Риму. Однією з найвідоміших реалізацій є гра Total War Rome 2, розроблена студією Creative Assembly, де було реалізовано поєднання стратегії в реальному часі та покрокової стратегії із сетингом Давнього Риму. Цей проект поєднує елементи стратегії в реальному часі з покроковою стратегічною картою, де гравець може обирати одну з багатьох фракцій на основі історичних сценаріїв. Гра пропонує масштабні битви з тисячами воїнів на екрані, детальну систему управління провінціями та дипломатії, а також історичні кампанії, засновані на реальних подіях.

Іншим цікавим прикладом є гра Ryse Son of Rome, створена компанією Crytek, де гравець бере під своє керування простого легіонера, котрий з часом проходить підвищення до командувача. На відміну від стратегічного жанру попереднього проекту, ця гра належить до категорії action-adventure з видом від третьої особи. Гравець керує римським легіонером, який проходить шлях від

звичайного солдата до командира високого рангу. Геймплей зосереджений на динамічних бойових сценах та системі швидкого реагування на дії противників. Гра відрізняється кінематографічною подачею та вражаючою графікою.

Аналізуючи ці реалізації, можна виділити як їхні сильні сторони, так і суттєві недоліки. Total War Rome 2 пропонує глибоку стратегічну складову та масштабність, проте висока складність механік може бути серйозним бар'єром для початківців, які лише знайомляться з історією Риму. Система управління вимагає значного часу на освоєння, що може відлякати користувачів. Великі битви потребують потужного апаратного забезпечення для комфортної гри, що робить цей продукт недоступним для користувачів із звичайними комп'ютерами. Крім того, історична достовірність іноді приноситься в жертву ігровому балансу. Гра також є комерційним продуктом з високою ціною.

Ryse Son of Rome зосереджується на індивідуальному досвіді одного воїна, що дозволяє краще відчувати атмосферу. Однак лінійність сюжету та обмежена варіативність геймплею значно знижують реіграбельність проекту. Гра також критикується за спрощену бойову систему, де після освоєння базових механік процес стає досить повторюваним. Освітній аспект присутній переважно у формі довідкової інформації, яка не інтегрована органічно в ігровий процес.

Крім того, дана реалізація підлягає обмеженням PG через присутність моментів жорстокості, неприхованої крові та насильства, що є перешкодою для розповсюдження серед молодшого покоління. Обидва проекти мають високі системні вимоги та є платними продуктами, що обмежує доступ широкої аудиторії студентів.

З точки зору освітньої цінності, існуючі реалізації не завжди дотримуються оптимального балансу між розважальною та навчальною складовою. Також варто відзначити відсутність проектів, орієнтованих на демонстрацію принципів об'єктно-орієнтованого програмування через історичну тематику. Більшість навчальних програм з ООП використовують абстрактні приклади, упускаючи можливість поєднати програмування з історією. Така комбінація могла б підвищити мотивацію студентів до вивчення обох дисциплін одночасно.

Враховуючи вищезазначені обмеження, моя реалізація є актуальною, оскільки пропонує безкоштовний та доступний інструмент для вивчення історії, який не вимагає потужного обладнання. Програма дозволить студентам бачити, як абстрактні класи відображають реальну ієрархію римського війська. Використання Java забезпечить платформонезалежність.

1.3 Розробка технічного завдання на роботу

В програмі на мові програмування Java повинно бути створено застосунок з графічним інтерфейсом. Клас мікрооб'єкта повинен містити не менше 4 елементів змінних і не менше 4 методів (окрім геттерів та сеттерів). Як мінімум одна змінна повинна бути типу `int`, одна – типу `double` і як мінімум одна – рядок. Зображення мікрооб'єкта: мінімум три графічних примітива, один з яких – текст (навіть в неактивному стані). Білий (=невидимий) графічний примітив не рахується. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання. Для класу мікрооб'єкта слід реалізувати глибинне копіювання. Зображення мікрооб'єкта повинно містити графічні примітиви, які показують стан елемента посилального типу, який був доданий до класу мікрооб'єкта, щоб зробити необхідним глибинне копіювання при клонуванні. При натискуванні лівої кнопки миші на мікрооб'єкті він повинен ставати активним/неактивним. В програмі повинна бути реалізована активація декількох об'єктів одночасно. Зображення активного мікрооб'єкта повинно відрізнятись від зображення неактивного хоча б одним графічним примітивом. Інтерфейс програми повинен сигналізувати про те, який саме мікрооб'єкт активував користувач. По вибору студента інформація про активний мікрооб'єкт/мікрооб'єкти може виводитись у рядку статусу або робочій області програми. Якщо користувач активував декілька мікрооб'єктів, то інформація про це також має якимось чином відображатися. При натискувань клавіш-стрілок активні об'єкти/об'єкт повинні рухатись у вікні програми. При натискуванні клавіши `Delete` активні об'єкти повинні знищуватись. Якщо активного об'єкта нема – клавіша, то клавіша `Delete`

ігнорується. Клавiша Esc повинна відмінати активацію об'єкта. По певній комбінації клавiш (наприклад Ctrl-C) повинно бути реалізоване копіювання активного мікрооб'єкта. При натискуванні правої кнопки миші на мікрооб'єкті повинно з'являтися діалогове вікно, яке дозволяє змінювати параметри вказаного мікрооб'єкта. Додати до проекту три або більше макрооб'єкта. Макрооб'єкти повинні мати можливість містити більше двох мікрооб'єктів одночасно. Додати команди рисування макрооб'єктів в програму. Не обов'язково мати можливості додавати/видаляти макрооб'єкти в програму. Цілком достатньо, якщо програма буде мати деяку кількість макрооб'єктів згенеровану при запуску програми і яка буде незмінною протягом роботи програми. Макрооб'єкти повинні мати певні позиції в універсальному об'єкті і повинні відображатись на екрані. Позиції макрооб'єктів всередині універсального об'єкта можуть залишатись незмінними протягом роботи програми. В універсальному об'єкті повинно міститись як мінімум один екземпляр кожного оголошеного в темі макрооб'єкта. Зображення кожного макрооб'єкту повинно складатись не менше чим з трьох графічних примітивів (лінія, прямокутник, коло, арка, полігон, текст, графічний образ та інших), при цьому обов'язково один з примітивів повинен бути текст. Білий (=невидимий) графічний примітив не рахується. Мікрооб'єкти можуть або належати одному або більшій кількості макрооб'єктів або не належати жодному. Глядачу має бути зрозуміло в чому полягає приналежність мікрооб'єкта макрооб'єкту, тобто чим вигляд (або поведінка) такого мікрооб'єкта відрізняється від поведінки мікрооб'єкта, який не належить жодному макрооб'єкту. Повинна обов'язково бути реалізована можливість вилучити мікрооб'єкт з всіх макрооб'єктів, в результаті чого він не буде належати жодному макрооб'єкту. Повинна обов'язково бути реалізована можливість зробити так, щоб активний мікрооб'єкт став належати одному з наявних макрооб'єктів. В зображенні макрооб'єкта обов'язково повинно виводитись кількість мікрооб'єктів, які йому належать. Повина бути побудована ієрархія класів мікрооб'єктів, яка містить як мінімум три рівня наслідування, які можуть інстанціюватись і зображатись на екрані. Зображення кожного рівня має відрізнятись від всіх інших хоча б одним

графічним примітивом. В побудованій ієрархії класів повинні бути продемонстровані: а) виклик конструктора базового класу; б) виклик метода базового класу; в) використання динамічного поліморфізму; г) використання статичного поліморфізму. Повинно бути продемонстровано використання ключового слова `instanceof`. Повинен бути реалізований автоматичний рух деяких мікрооб'єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою (це має бути помітно візуально в тому сенсі, що або щось змінюється у їх зображенні або у характері їх руху); 2) деякі мікрооб'єкти заходять в макрооб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Під характером руху мається на увазі не просто траєкторія, а те, в який спосіб вони взаємодіють з макрооб'єктами та іншими мікрооб'єктами. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач (будь яким способом). Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) на мінікарті повинна бути позначена видима область універсального об'єкта; 2) рух видимої області універсального об'єкта відповідно змінює мінікарту; 3) макрооб'єкти та мікрооб'єкти позначені на мінікарті; 4) рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті; 5) натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на карті. При серіалізації/де-серіалізації обов'язково повинні використовуватись діалогові вікна, щоб запитати у користувача ім'я файлу та його розташування на диску (наприклад можна використовувати системне діалогове вікно `FileChooser`). Продемонструвати використання лямбда-виразу. Продемонструвати використання операція з потоком `Stream`. При натискуванні клавіші `Insert` повинно з'являтися діалогове вікно, яке повинно визначати

параметри створюваного мікрооб'єкта. В нього повинна бути присутня можливість обирати до якого з класів нащадків у ієрархії наслідування він належить. Крім керуючого елемента Button у діалоговому вікні також повинні бути використані як мінімум три з наступного списку: TextField, CheckBox, {ComboBox || ListView || ChoiceBox}, RadioButton.

Повинен бути реалізований наступний запит: знайти мікрооб'єкт з вказаними користувачем параметрами, в результаті якого програма шукає мікрооб'єкт і видає на екран: його місце розташування, приналежність цього мікрооб'єкта до макрооб'єктів (чи зазначає, що мікрооб'єкт не належить жодному макрооб'єкту). Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які належать вказаному макрооб'єкту. Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які не належать жодному макрооб'єкту. Повинен бути реалізований наступний запит: встановити критерій сортування мікрооб'єктів у запитах, які виводять переліки. Критеріїв сортування повинно бути не менше трьох, наприклад: Ім'я_студента, Середній_бал, Кількість_Оцінок_4_та_вище.

Повинні бути реалізовані як мінімум три запити по підрахуванню мікрооб'єктів. Наприклад а) кількість активних мікрооб'єктів, кількість мікрооб'єктів з рівнем життя більше половини, кількість мікрооб'єктів, які знаходяться в правій половині екрану тощо.

Для нормального виконання програми необхідно наступне апаратне забезпечення:

1. Комп'ютер серії IBM PC з частотою 3200 МГц і вище.
2. Операційні системи: Windows Vista/7/10/11; Linux; Mac OS.
3. 32 Gb оперативної пам'яті DDR4.
4. Графічний адаптер SVGA (Super Video Graphic Adapter).
5. Відео-карта об'ємом пам'яті не менше 8 Gb.
6. Клавіатура IBM, миша.

Розмір дискового простору, який потрібен для коректної роботи програми: 30 Mb.

Розмір оперативної пам'яті, що займає програма: 300 Mb.

1.4 Обґрунтування вибору мови програмування

Вибір мови програмування є критично важливим рішенням на етапі проектування будь-якого програмного продукту. Для реалізації проекту на тему Roman Conquest необхідно порівняти три основні мови програмування: C++, C# та Java, і довести, що для даної програми Java є оптимальним вибором. Кожна з цих мов має свої особливості, переваги та недоліки, які необхідно детально проаналізувати.

C++ є потужною мовою програмування, яка розвивалася з мови C шляхом додавання об'єктно-орієнтованих можливостей. Ця мова надає програмісту максимальний контроль над апаратними ресурсами, дозволяє працювати безпосередньо з пам'яттю через покажчики та пропонує високу продуктивність. Програми на C++ компілюються в машинний код для конкретної платформи, що забезпечує швидкість виконання, але вимагає окремої компіляції для кожної операційної системи.

C# розроблена компанією Microsoft як частина платформи .NET Framework та має багато спільного з Java за синтаксисом та концепціями. Ця мова тісно інтегрована з екосистемою Microsoft, що робить її відмінним вибором для розробки додатків під Windows. Протягом останніх років підтримка інших платформ покращилася завдяки проекту .NET Core, але історично C# мала обмеження щодо кросплатформенності.

Java є об'єктно-орієнтованою мовою програмування, розробленою компанією Sun Microsystems у середині дев'яностих років. Головною філософією Java є принцип «напиши один раз, запускай скрізь», що досягається завдяки використанню віртуальної машини Java. Програма, написана на Java, компілюється в байт-код, який потім інтерпретується віртуальною машиною незалежно від операційної системи. Це означає, що один і той же виконуваний файл може працювати на Windows, macOS, Linux без перекомпіляції.

Для детального порівняння цих мов доцільно розглянути їхні характеристики, наведені в таблиці 1.1.

Таблиця 1.1 – Порівняння мов програмування C++, C# та Java

Характеристика	C++	C#	Java
Платформонезалежність	Потребує окремої компіляції для кожної платформи	Обмежена, покращена в .NET Core	Повна підтримка через JVM
Управління пам'яттю	Ручне через покажчики	Автоматичне збирання сміття	Автоматичне збирання сміття
Швидкість розробки	Середня через складність синтаксису	Висока завдяки інструментам Microsoft	Висока завдяки бібліотекам
Складність вивчення	Висока через низькорівневі концепції	Середня, схожа на Java	Середня, зрозумілий синтаксис
Графічні бібліотеки	Qt, wxWidgets, SFML	WPF, Windows Forms	JavaFX, Swing
Багатопотоковість	Складніша реалізація	Вбудована підтримка	Вбудована підтримка

Аналізуючи специфіку проекту на тему Roman Conquest, можна виділити кілька ключових вимог. По-перше, програма має бути доступною для максимально широкої аудиторії, включаючи користувачів різних операційних систем. Студенти можуть працювати на комп'ютерах з Windows, macOS або Linux, тому важливо

забезпечити однакову функціональність на всіх платформах. По-друге, для навчального проекту важлива простота та зрозумілість коду. Java пропонує чистий синтаксис, де кожен клас зберігається в окремому файлі, що полегшує навігацію. C++ має складнішу структуру з заголовковими файлами, що може заплутати початківців.

По-третє, автоматичне управління пам'яттю є важливим фактором для навчального проекту. У Java та C# програмісту не потрібно вручну виділяти та звільняти пам'ять, оскільки це робить вбудований збирач сміття. У C++ необхідно самостійно контролювати час життя об'єктів, що вимагає досвіду. Для студентів, які починають вивчати об'єктно-орієнтоване програмування, автоматичне управління пам'яттю дозволяє зосередитися на логіці програми та принципах ООП. По-четверте, наявність потужної стандартної бібліотеки значно прискорює розробку. Java надає величезний набір готових класів для роботи з колекціями, файлами, графікою. Бібліотека JavaFX пропонує сучасний підхід до створення графічних інтерфейсів з підтримкою анімації.

По-п'яте, підтримка багатопотоковості є важливою для створення динамічної симуляції, де різні об'єкти повинні функціонувати одночасно. Java має вбудовану підтримку багатопотокового програмування з класами Thread, Runnable та пакетом java.util.concurrent. Це дозволяє реалізувати паралельне виконання логіки різних ігрових об'єктів без ускладнення коду. Що стосується C#, хоча ця мова має багато спільного з Java, вона традиційно орієнтована на екосистему Windows. Java залишається більш перевіреним рішенням для створення додатків, які працюють на різних платформах. Також варто враховувати, що багато навчальних закладів використовують Java як основну мову для викладання об'єктно-орієнтованого програмування.

Популярність мови також відіграє роль у виборі. Згідно з рейтингом ТЮВЕ, який відстежує популярність мов програмування на основі кількості пошукових запитів, Java стабільно входить до трійки найпопулярніших мов програмування у світі. Це означає наявність великої спільноти розробників, величезної кількості навчальних матеріалів, готових бібліотек та фреймворків, а також активну

підтримку мови. Враховуючи всі вищезазначені фактори, Java є оптимальним вибором для реалізації проекту на тему Roman Conquest. Ця мова забезпечує необхідну платформонезалежність, має зрозумілий синтаксис для навчальних цілей, пропонує автоматичне управління пам'яттю, містить потужну бібліотеку JavaFX для створення графічного інтерфейсу, надає засоби для багатопотокового програмування та підтримується великою спільнотою. Усі ці переваги роблять Java ідеальним інструментом для створення освітнього програмного продукту.

1.5 Висновки

У першому розділі було проаналізовано предметну область проекту, що охоплює військову організацію Давнього Риму з її ієрархічною структурою. Розглянуто мікрооб'єкти (воїни, центуріони, преторіанці) та макрооб'єкти (джерела руди, башти, бази). Комплексний аналіз існуючих реалізацій виявив їхні недоліки: високі системні вимоги та значну вартість. Порівняння мов програмування C++, C# та Java дозволило обґрунтувати вибір Java як оптимального інструменту завдяки платформонезалежності, зрозумілому синтаксису та потужній бібліотеці JavaFX.

2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

2.1 Модель графічного відображення

Кожен мікрооб'єкт складається з власної PNG-картинки, прямокутника, що відображає рівень очок здоров'я мікрооб'єкта, назва мікрооб'єкта та маркер команди мікрооб'єкта.

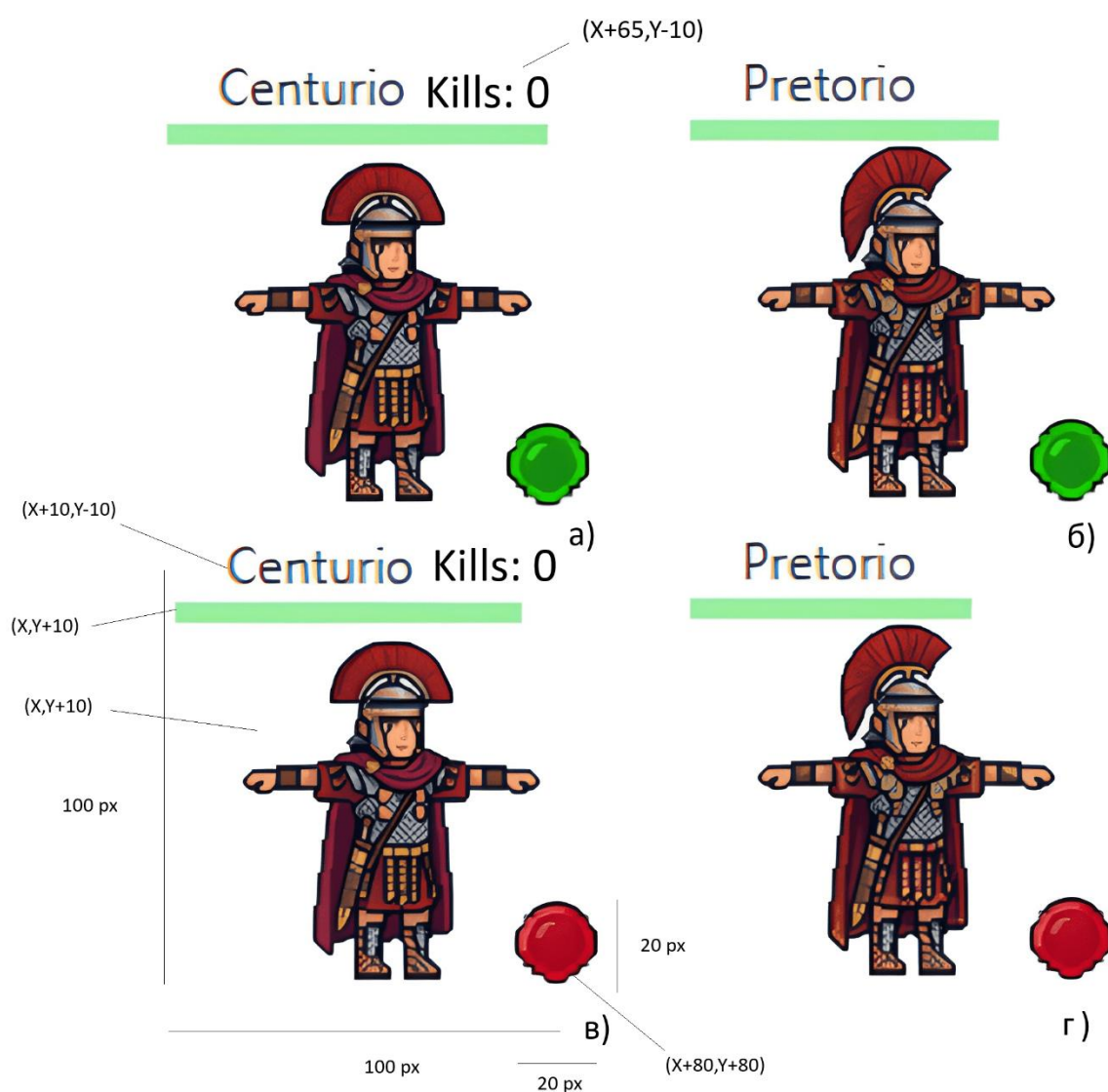


Рисунок 2.1 – Зображення мікрооб'єктів: а) Центуріон за команду союзників; б) Преторіанець за команду союзників; в) Центуріон за команду противників; г) Преторіанець за команду противників

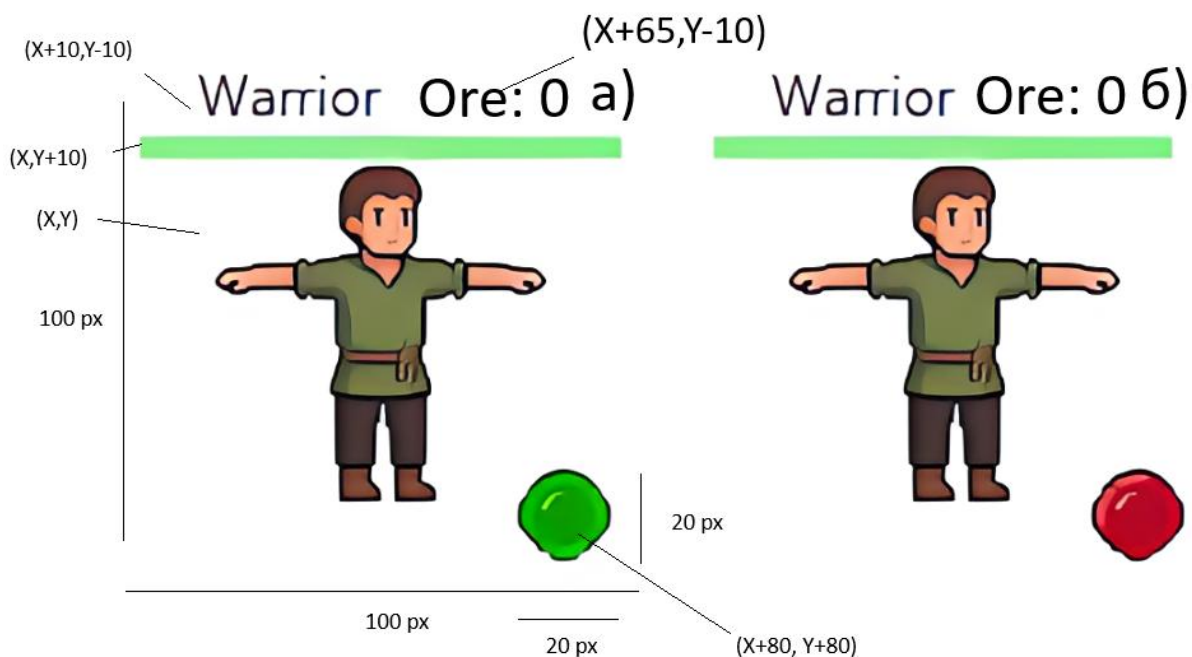


Рисунок 2.2 – Зображення мікрооб’єктів: а) Воїн за команду союзників; б) Воїн за команду противників

Для вирахування мікрооб’єктів за основу було взято початкове положення зображення X та Y (див. рисунок 2.1 ,2.2), які є приватними полями у батьківському класі мікрооб’єкта. Коефіцієнти були підібрані так, щоб було чітко видно кожен елемент мікрооб’єкта.

Кожен макрооб’єкт складається з власної PNG-картинки, прямокутника, що відображає рівень міцності споруди, назва макрооб’єкта, лічильник мікрооб’єктів у реальному часі, що знаходяться в даному макрооб’єкті та маркер, що показую команду до якої належить макрооб’єкт. Всі примітивні графічні елементи зображенні стосовно початкових координат (рисунок 2.3 – 2.5).

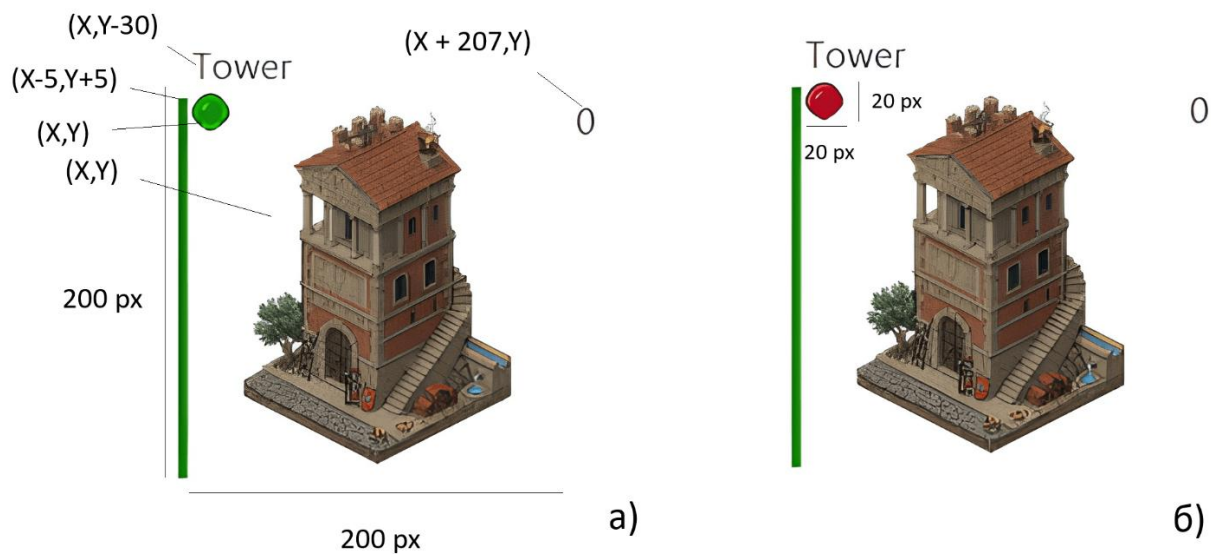


Рисунок 2.3 – Зображення макрооб'єктів: а) Башта за команду союзників; б) Башта за команду противників

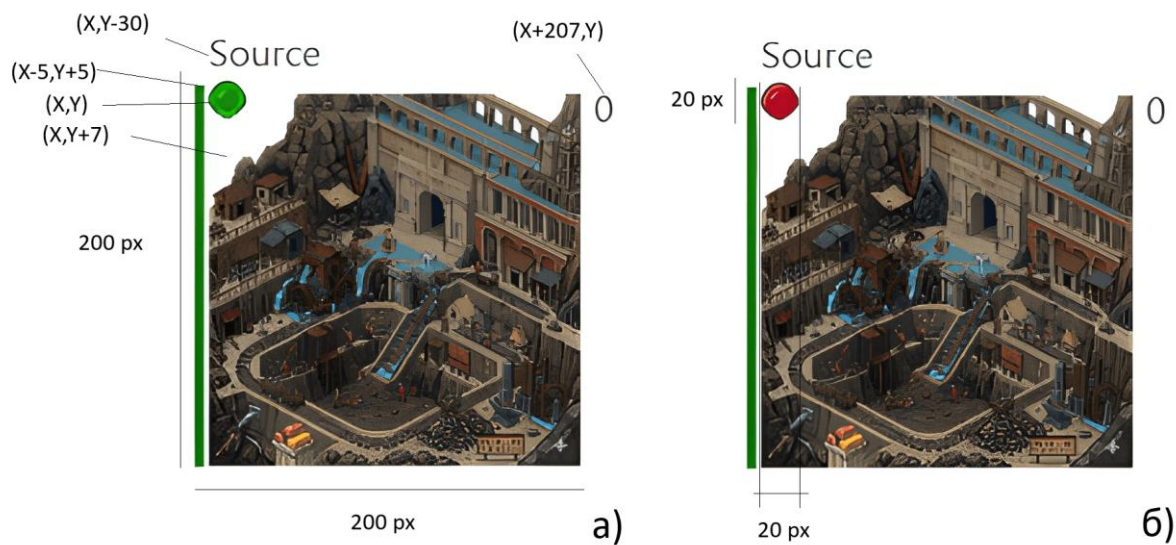


Рисунок 2.4 – Зображення макрооб'єктів: а) Джерело руди за команду союзників; б) Джерело руди за команду противників

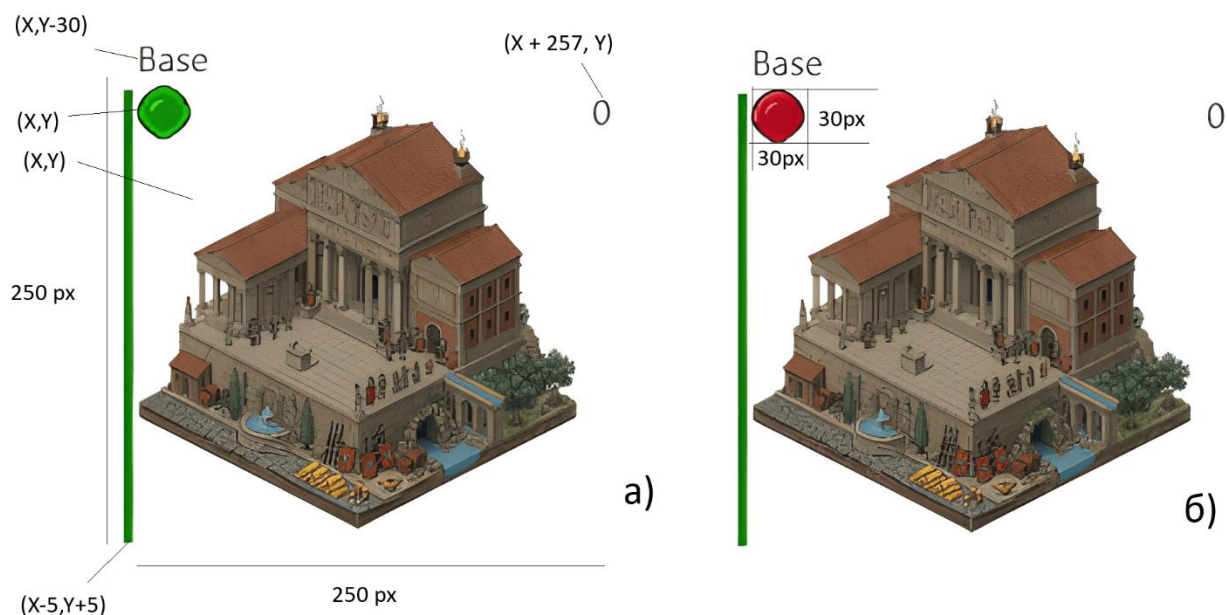


Рисунок 2.5 – Зображення макрооб'єктів: а) База за команду союзників; б) База за команду противників

Мікрооб'єкти складаються з трьох основних графічних примітивів, а також з 1 динамічного графічного примітива для класу Warrior та Centurio, що відображає кількість необхідної руди або кількість необхідних вбивств щоб отримати наступний рівень ієрархії для поточного мікрооб'єкта.

2.2 Графічні процедури підсистеми графічного відображення

Для створення зображень мікро- та макрооб'єктів у програмі було використано графічні засоби бібліотеки JavaFX. Основними використаними графічними примітивами були:

`Group()` – конструктор, що створює групу, яка може містити у собі деяку кількість елементів класу `Node` для їх спільного розміщення на головний екран програми.

`Scene(Parent root, double width, double height)` – створює основну сцену розміром 1080×700 пікселів, яка містить `Group` з усіма графічними об'єктами.

`Image(String url)` – завантажує картинку із переданого URL-посилання або шляху у пам'яті комп'ютера.

`ImageView(Image image)` – конструктор елементу-картинки, який є наслідником класу `Node`, що дозволяє розміщати його на екрані.

`Label(String text)` – створює текстову мітку із заданим текстом.

`Line()` – створює лінійний елемент для відображення полоски здоров'я.

`Rectangle(double x, double y, double width, double height)` – створює прямокутник з координатами верхнього лівого кута (x, y), шириною та висотою.

`setCoordinates()` – переписаний методу класу `Unit`, який синхронізує всі графічні елементи (`Label`, `Line`, `ImageView`, `Rectangle`) з логічними координатами юнита.

`setPosition(double newX, double newY)` – встановлює нову позицію юнита на екрані.

`getBoundsInParent().contains(double x, double y)` – метод `ImageView`, який перевіряє, чи знаходиться точка з координатами (x, y) всередині меж зображення.

`AnimationTimer()` – конструктор елементу, який містить методи `start()`, `stop()` та переписаний метод `handle(long now)`.

3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

Діагра́ма класів (англ. class diagram) — статичне представлення структури моделі. Відображає статичні (декларативні) елементи, такі як: класи, типи даних, їх зміст та відношення.

3.1 Діаграма наслідування

Наслідування – це один з принципів об'єктно-орієнтованого програмування, який дозволяє описати новий клас на основі існуючого. При цьому властивості і функції базового класу успадковуються нащадком (рисунок 3.1).

Базовий рівень (Unit) є універсальним класом для всіх класів мікрооб'єктів у програмі. Він реалізує інтерфейс Cloneable та містить основні поля та методи, які характеризують загальні властивості мікрооб'єкта, такі як: health – рівень здоров'я; isSpawned – флаг спавну; isDead – флаг смерті; damage – урон; team – приналежність до команди; inventor – контейнер інвентарю; baseHealth – базова кількість здоров'я; baseDamage – базовий урон; numObjects – статичний лічильник об'єктів класу; labelName – надпис з ім'ям (перший графічний примітив); life: Line – смужка здоров'я (другий графічний примітив); image: ImageView – png картинка юніта (третій графічний примітив); rectActive: Rectangle – область активності мікрооб'єкта; imageMark – графічний показник команди; mainWeaponImage – графічне представлення головної зброї (четвертий графічний примітив); x, y – координати мікрооб'єкта на екрані; moveSpeed = 0.005 – швидкість руху; lastAttackTime – час останньої атаки; ATTACK_COOLDOWN = 1000 – час перезавантаження атаки; а також деякі інші технічні змінні для графіки та логіки.

«V» перед функціями означає: «віртуальна». Основні методи Unit включають: V void takeDamage() – обробляє отримання шкоди; V void setPosition(double x, double y) – встановлює позицію юніта; V void resurrect() – додає графічні елементи юніта на екран; V void logic() – встановлює поведінку конкретного типу юніта; V void attack() – обробляє атаку юніта на ворогів; V void

`moveTo(double newX, double newY)` – метод переміщення юніта та всіх його графічних примітивів; `V void removeUnitFromGame()` – видалення юніта з гри та очищення ресурсів.

У програмі реалізовано наслідування чотирьох рівнів (рисунок 3.1). Перший рівень (Warrior) наслідує Unit і розширює функціональність полями та методами для збору ресурсів (руди). Основні атрибути: `name = "Warrior"` – ім'я (статична константа); `MAX_HEALTH = 100.0` – максимальне здоров'я (константа); `oreAmount` – поточна кількість зібраної руди; `activeOre` – активна руда під час збирання (до 10); `ORE_COOLDOWN = 1000` – час перезагрузки збору руди; `lastOreTime` – час останнього збору; `collectingOre` – флаг процесу збору руди; `deliveringOre` – флаг процесу доставки руди; `COMBAT_PRIORITY_RADIUS = 180.0` – радіус пріоритету боїв з ворогами; `oreCountLabel` – графічний показник активної кількості руди (п'ятий графічний примітив); а також деякі інші технічні змінні. Основні методи Warrior: `V void logic()` – логіка руху, пошуку ворогів, збору та доставки руди; `V void doOre()` – обробляє цикл збору руди зі Source та доставки до Base; `V void promotion()` – промоція Warrior до Centurio при наборі 50 одиниць руди; `protected void setOreCount(int oreCount)` – встановлює активну кількість руди; `public double getOre()` – повертає поточну кількість накопленої руди.

Другий рівень (Centurio) наслідує Warrior і розширює можливості додаванням обліку вбивств та лікування союзних будівель. Основні атрибути: `name = "Centurio"` – ім'я (статична константа); `MAX_HEALTH = 120.0` – максимальне здоров'я (константа); `healNum = 10.0` – величина лікування будівлі за один раз; `HEAL_COOLDOWN = 2000` – час перезагрузки лікування; `lastHealTime` – час останнього лікування; `killCount` – лічильник вбитих ворожих юнітів; `killCountLabel` – графічний показник кількості вбивств (шостий графічний примітив); `countedKills: HashSet` – множина відслідкованих вбитих юнітів для уникнення подвійного рахунку; а також деякі інші технічні змінні. Основні методи Centurio: `V void attack()` – атака з обліком вбитих ворогів та збільшенням лічильника `killCount`; `V void logic()` – розширена логіка вибору цілей (враги, будівлі, союзні будівлі для лікування, з пріоритезацією союзних будівель що потребують лікування); `V void promotion()` –

промоція Centurio до Pretorio при наборі 5 вбивств; `private void heal(World target)` – лікування союзної будівлі якщо знаходиться в зоні 100 пікселів; `public int getKillCount()` – повертає кількість вбивств.

Третій рівень (Pretorio) наслідує Centurio і додає здатність групового лікування для союзних юнітів в радіусі ефекту. Основні атрибути: `name = "Pretorio"` – ім'я (статична константа); `MAX_HEALTH = 150.0` – максимальне здоров'я (константа); `healNum = 5.0` – величина лікування кожного союзного юніта за один раз; `areaOfEffect: Circle` – графічне представлення зони радіусу 150 пікселів для групового лікування (сьомий графічний примітив); `HEALAREA_COOLDOWN = 1000` – час перезагрузки групового лікування; `lastHealAreaTime` – час останнього групового лікування; а також деякі інші технічні змінні. Основні методи Pretorio: `V void logic()` – розширена логіка вибору цілей та групового лікування; `V void healInArea()` – лікування всіх союзних юнітів (крім себе) в радіусі 150 пікселів, що мають здоров'я > 0 ; `V void flipActivation()` – перемикає активності юніта з показом/приховуванням зони ефекту; `V void resurrect()` – додає зону ефекту на екран; `V void setCoordinates()` – оновлює позицію зони ефекту. Рівні при такому розширюються – мікрооб'єкт отримує максимальне здоров'я 150 та нові здатності групового лікування союзних юнітів.

3.2 Діаграма агрегації

Відношення агрегації описує зв'язок між об'єктом-контейнером та його елементами, які можна динамічно додавати або вилучати. Як правило, таке співвідношення має формат 1:N. Суть агрегації полягає в тому, що один об'єкт зберігає посилання на інший, проте їхні життєві цикли не пов'язані — обидва можуть існувати як самостійні сутності незалежно один від одного. У даній програмі прикладами агрегації слугують зв'язки `Base – Unit`, `Tower – Unit`, `Source – Unit` та `Unit – inventor` (рисунок 3.2). Кожна із зазначених сутностей виступає контейнером для іншої, вміщуючи в собі елементи певного класу чи компонента.

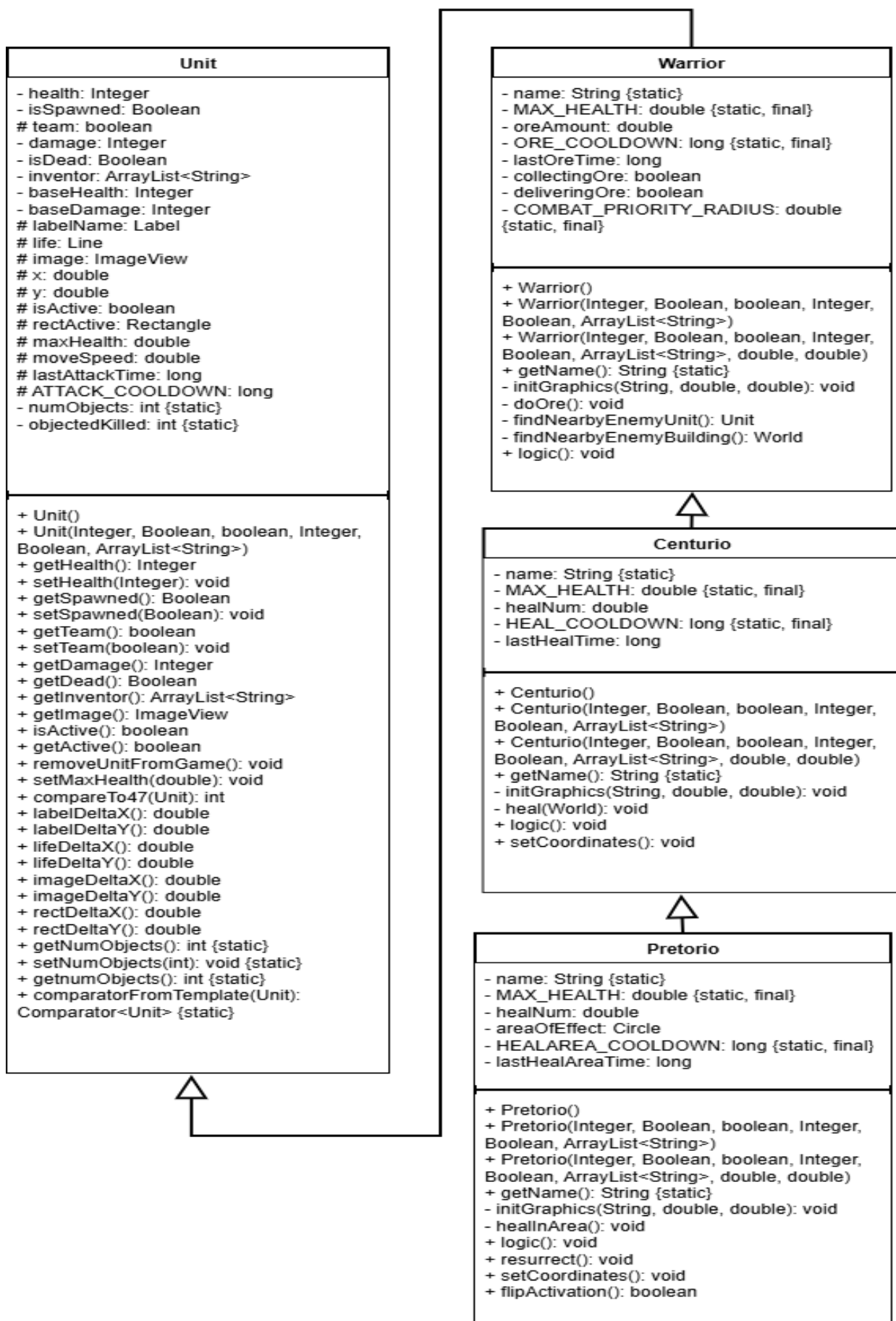


Рисунок 3.1 – Діаграма наслідування

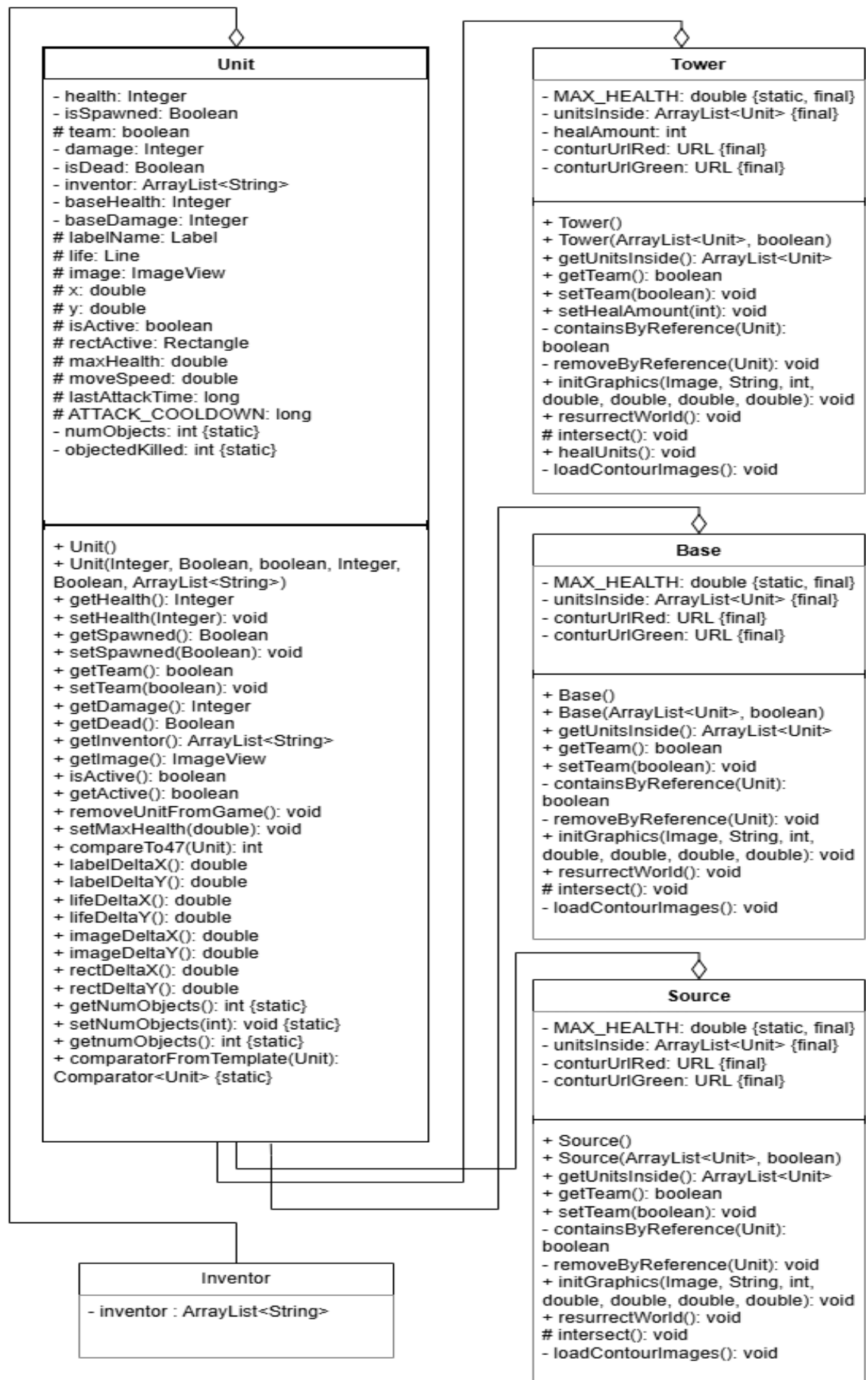


Рисунок 3.2 – Діаграма агрегації

В програмі співвідношення між класом мікрооб'єкта та макрооб'єктів є динамічними, мікрооб'єкти не можуть належати декільком мікрооб'єктам одночасно.

3.3 Діаграма композиції

Суть композиції абсолютно протилежна суті агрегації. Вона полягає у тому, що певний об'єкт є невід'ємною частиною іншого, який в свою чергу, не може існувати без попереднього. У програмі композиція реалізована у зв'язку мікрооб'єкт та його інвентар; мікрооб'єкт та його `labelName`; мікрооб'єкт та його показник здоров'я; мікрооб'єкт та його `image`; мікрооб'єкт та його `rectActive` (`Rectangle`) (рисунок 3.3). Неможливо створити мікрооб'єкт без перелічених параметрів.

3.4 Діаграма асоціації

Асоціація являє собою вид взаємозв'язку, при якому об'єкт містить у собі посилання на менший, службовий об'єкт. Це дозволяє одному об'єкту звертатися до іншого та використовувати його для досягнення визначених цілей.

В даній програмі прикладом асоціації є взаємодія між класом башти (`Tower`) та мікрооб'єкта (`Unit`) (рисунок 3.4). Клас макрооб'єкта `Tower` взаємодіє з кожним мікрооб'єктом, що належить до макрооб'єкта, шляхом зміни здоров'я в залежності від команди макрооб'єкта. При належності мікрооб'єкта до протилежної команди, здоров'я мікрооб'єкта буде зменшуватися протягом часу перебування в макрооб'єкті.

3.5 Діаграма кооперації

Кооперація (*collaboration*) — це специфікація групи об'єктів, які спільно взаємодіють для реалізації певних варіантів використання в загальному контексті

системи, що моделюється. Її головна мета полягає у деталізації особливостей реалізації цих варіантів використання або найбільш значущих системних операцій. Структура поведінки, застосована в розробленій програмі, визначає процес обробки повідомлень під час її виконання (рисунок 3.5).

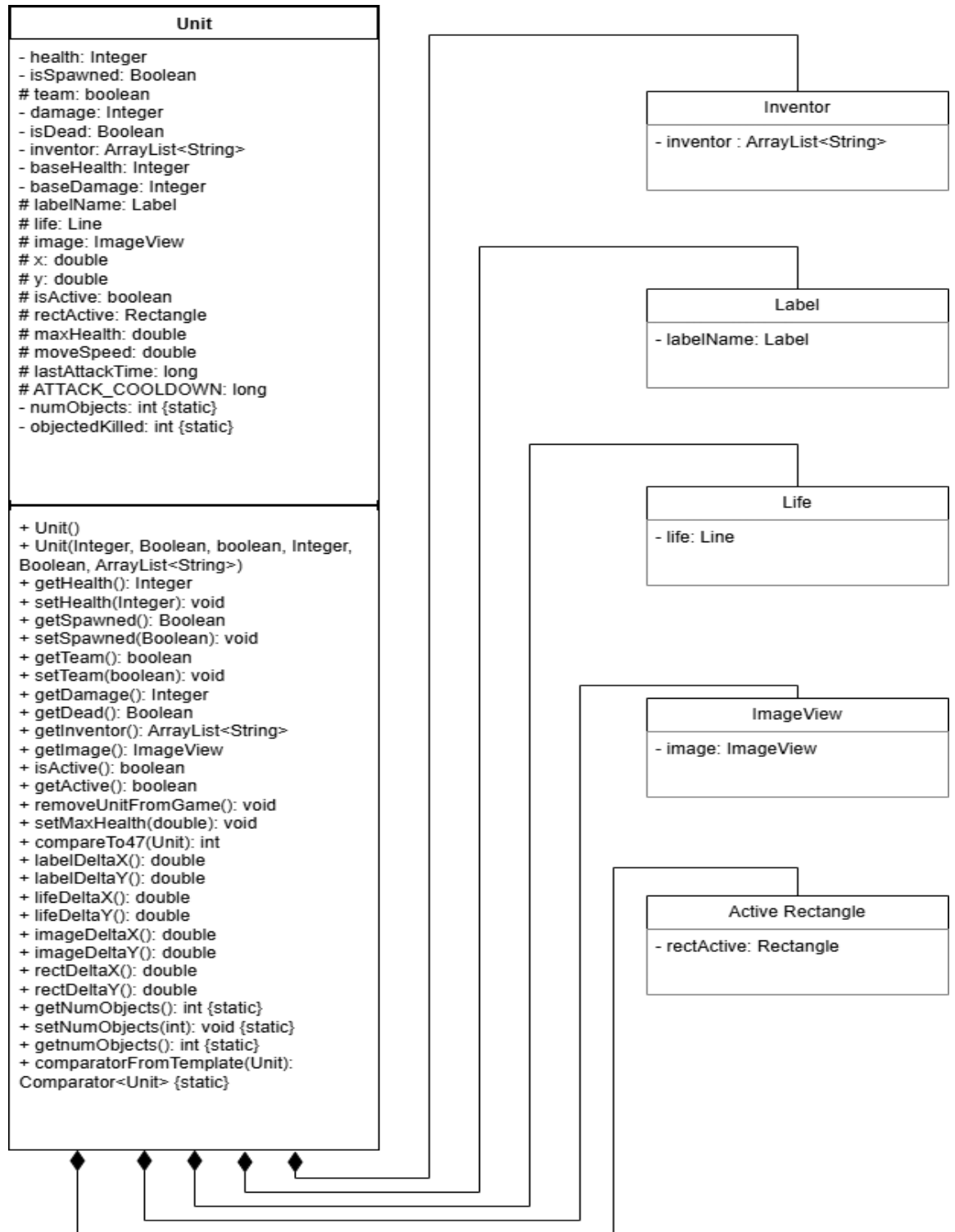


Рисунок 3.3 – Діаграма композиції

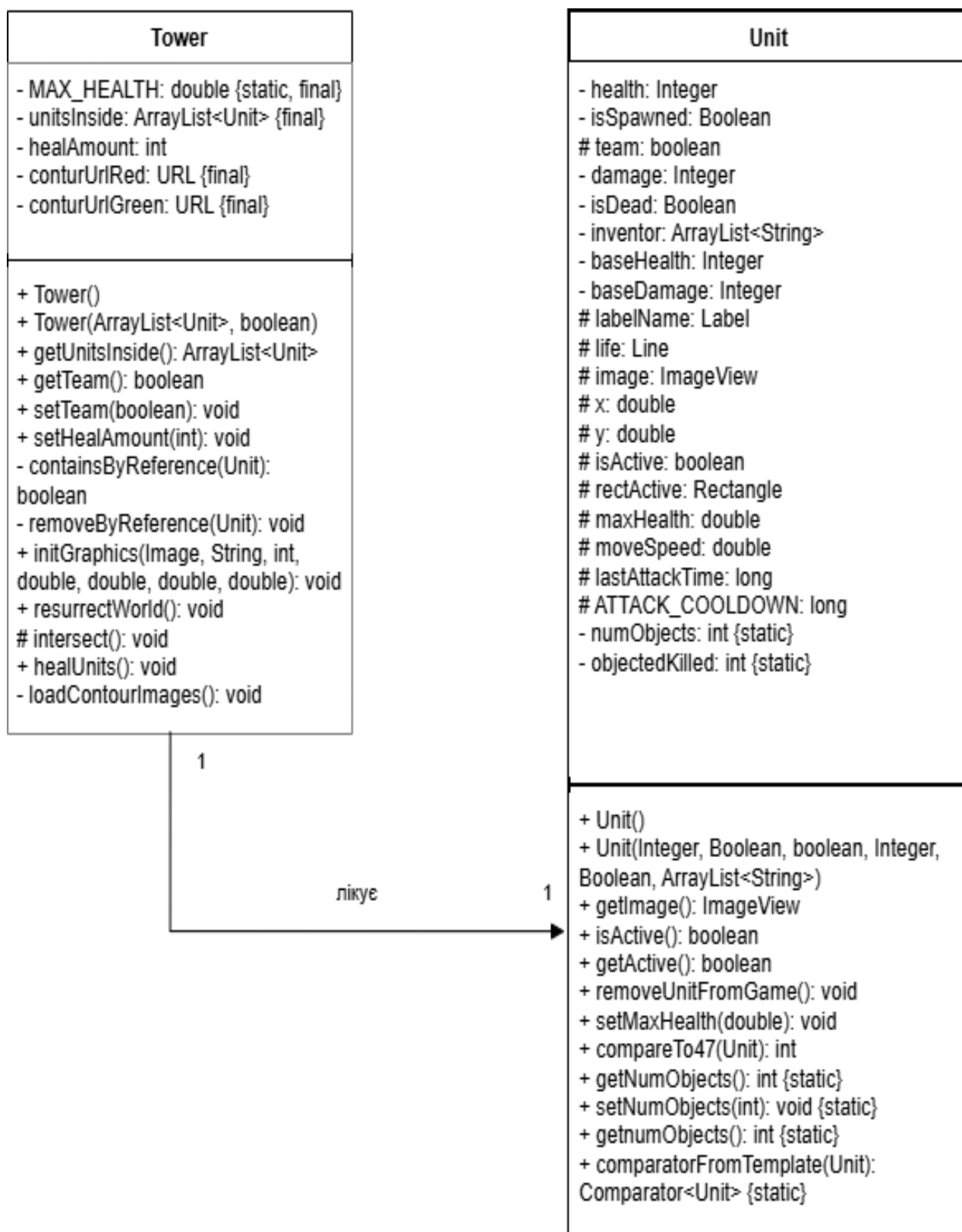


Рисунок 3.4 – Діаграма асоціації

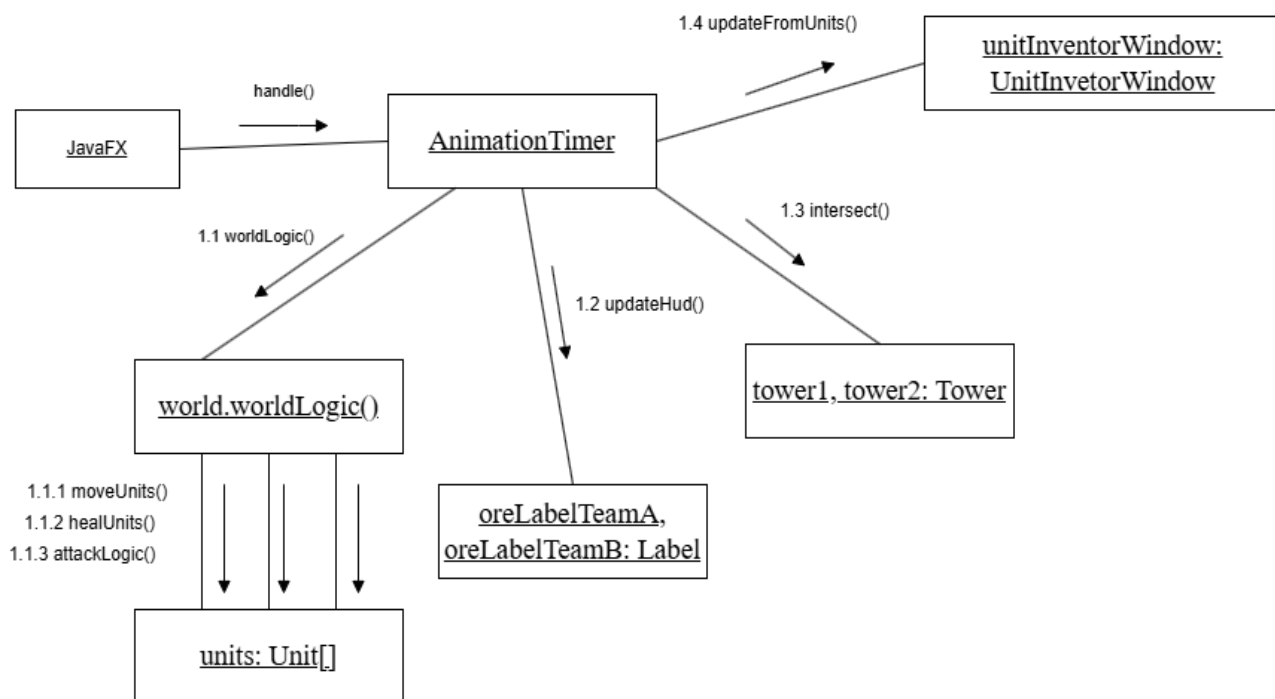


Рисунок 3.5 – Діаграма кооперації

Діаграма кооперації (див. рисунок 3.5) ілюструє перелік методів та класів що входять до кооперації протягом використання програми.

3.6 Діаграма послідовності

Діаграма послідовності (Sequence Diagram) — це різновид поведінкової діаграми UML, що відображає хронологічну послідовність взаємодії об'єктів системи в межах певного заданого сценарію. Вона наочно демонструє екземпляри об'єктів та процес їхньої комунікації один з одним через обмін повідомленнями в часі. Діаграми послідовності активно використовуються для візуалізації складної бізнес-логіки, життєвих циклів та механізмів обміну даними між компонентами.

Зокрема, розроблена діаграма дозволяє зрозуміти, як ігрові мікрооб'єкти взаємодіють один з одним та з ігровим середовищем, і що є безпосереднім наслідком їхньої взаємодії (рисунок 3.6). Вона ілюструє багатокроковий сценарій еволюції (покращення) бойових одиниць.

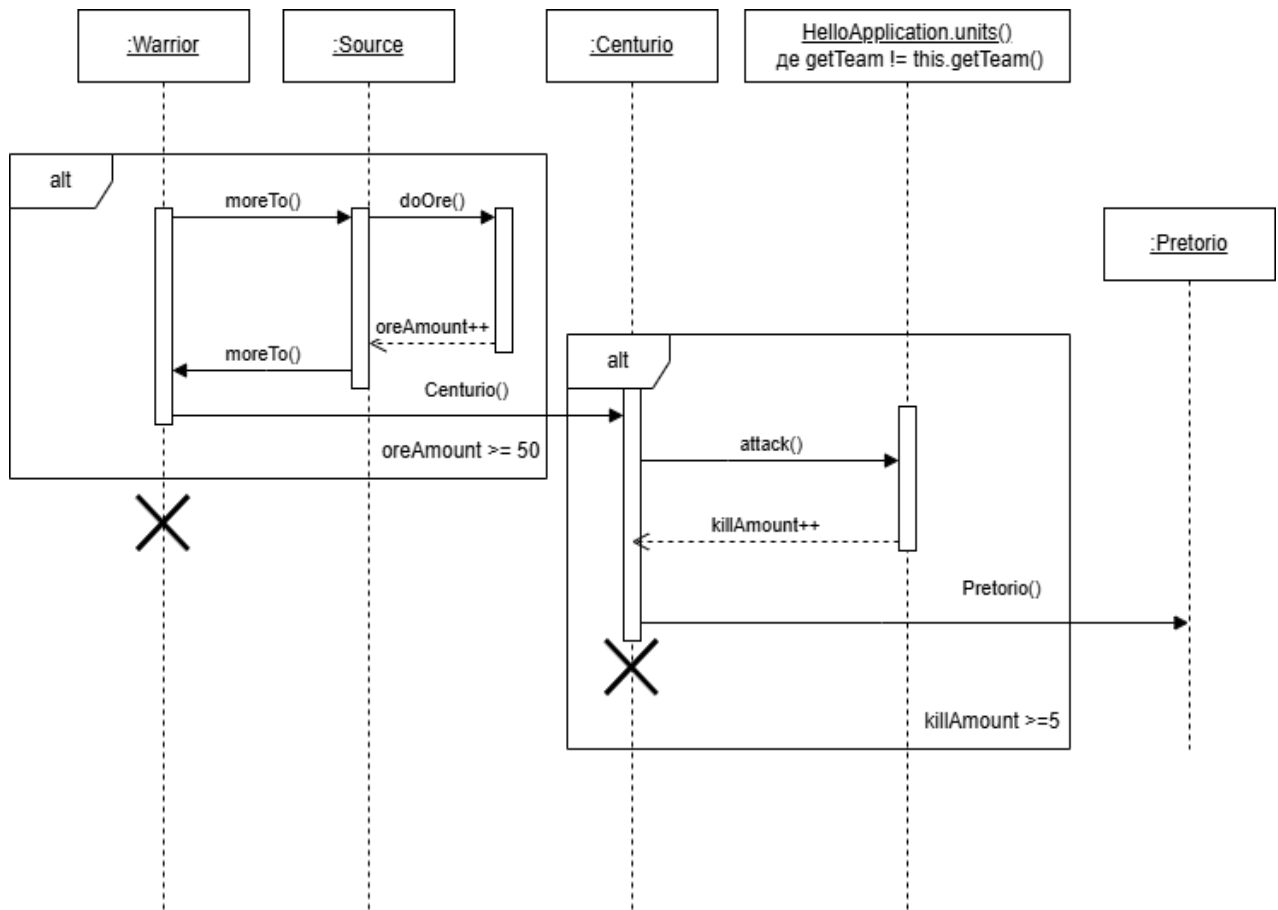


Рисунок 3.6 – Діаграма послідовності

Продемонстрована взаємодія мікрооб'єкта Warrior (див. рисунок 3.6), який протягом програми при певних умовах переходить до наступного в ієрархії класу мікрооб'єкта виконуючи певні дії, що дозволяють йому підвищити свою кваліфікацію.

3.7 Діаграма стану

Діаграма станів (State Diagram) — це поведінкова діаграма UML, що візуалізує динамічну поведінку об'єкта, його можливі стани та переходи між ними під впливом певних умов. Вона дозволяє детально простежити реакцію системи на різноманітні чинники під час її життєвого циклу (рисунок 3.7).

Життєвий цикл розпочинається зі створення базового юніта класу Warrior, головним завданням якого є накопичення ресурсів. Об'єкт циклічно виконує метод

doOre(), доки кількість зібраної руди не досягне необхідного мінімуму (oreAmount >= 50). Після успішного виконання цієї умови генерується подія promotion(), яка ініціює перехід об'єкта до вдосконаленого класу Centurio. Відтепер його ціль змінюється на здобуття бойового досвіду: юніт циклічно викликає метод attack() та взаємодіє з об'єктами колекції HelloApplication.units(), поки не здійснить п'ять знищень ворогів (killAmount >= 5).

Досягнення цієї бойової цілі викликає новий тригер promotion(), що трансформує об'єкт у фінальний елітний клас — Pretorio. Цей юніт продовжує вести бій аж до знищення бази супротивника, після чого система переходить до кінцевого успішного стану «Перемога!».

Водночас важливою умовою є те, що на будь-якому етапі еволюції (Warrior, Centurio чи Pretorio) система безперервно перевіряє рівень здоров'я юніта. Якщо цей показник падає до критичної позначки (health <= 0), відбувається негайний перехід до термінального стану, що символізує загибель та вилучення мікрооб'єкта з гри. Таким чином, діаграма комплексно описує як поетапний розвиток бойових одиниць, так і всі можливі сценарії завершення їхнього життєвого циклу.

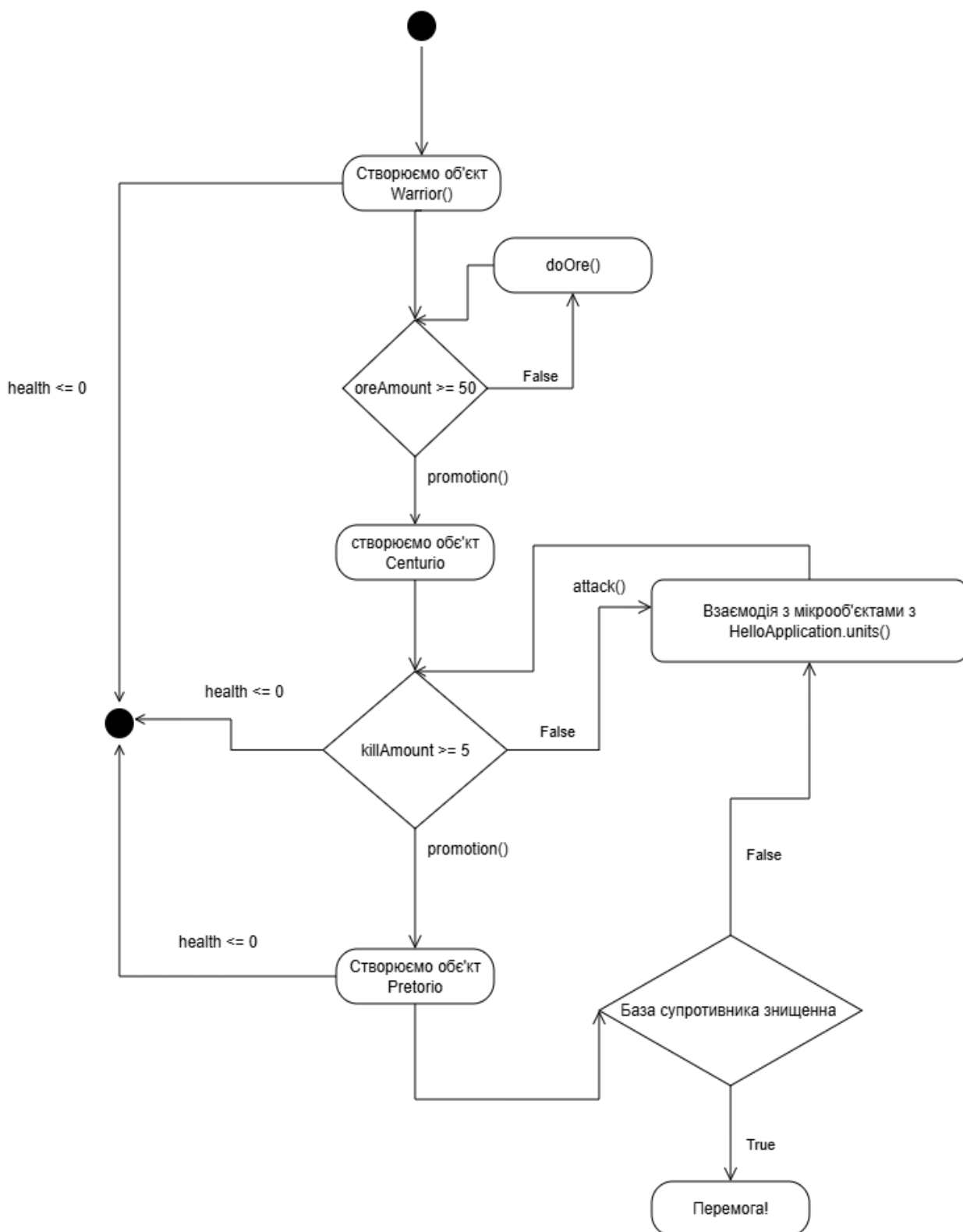


Рисунок 3.7 – Діаграма станів

Програма закінчується при умові, що база супротивника або мікрооб'єкту буде знищена мікрооб'єктом ворожої команди.